

UV Package manager

UV can:

- Install Python versions
- create virtual environments
- install dependencies
- manage lockfiles
- run Python scripts/tools
- publish packages

It is simpler, faster, more reliable, more modern

Especially for:

- AI/ML project
- backend APIs
- Automation
- Modern dev environment

uv sync

`uv sync` installs and synchronizes your project's dependencies based on the lockfile (`uv.lock`).

Specifically, it:

1. **Reads the lockfile** — uses `uv.lock` (not just `pyproject.toml`) to ensure exact, reproducible versions are installed.
2. **Creates/updates the virtual environment** — sets up `.venv` in your project directory if it doesn't exist.
3. **Installs missing packages** — adds anything in the lockfile that isn't already installed.
4. **Removes extra packages** — uninstalls anything in the environment that *isn't* in the lockfile, keeping the env clean.
5. **Installs the project itself** — by default, also installs your project package in editable mode.

Common flags:

- `--frozen` — fail if the lockfile is out of date (useful in CI)
- `--no-dev` — skip dev dependencies
- `--extra <name>` — include optional dependency groups
- `--all-extras` — include all optional extras

- `--no-install-project` — sync deps only, don't install the project itself

Think of it like `pip install -r requirements.txt` + cleanup, but powered by a lockfile for reproducibility.

uv.lock

A `uv.lock` file is a **lockfile** — it records the exact versions of every dependency (and their sub-dependencies) that were resolved for your project.

Its main purposes:

- **Reproducibility** — ensures everyone on the team (and CI/CD) installs the *exact same* package versions, not just "compatible" ones
- **Dependency pinning** — even if a package releases a new version, `uv sync` will still install what's in the lock file
- **Speed** — uv skips re-resolving dependencies since the work is already done
- **Auditing** — you can see every transitive dependency your project pulls in

It's similar to:

- `package-lock.json` in Node/npm
- `Pipfile.lock` in pipenv
- `poetry.lock` in Poetry

You should **commit it to git** so your whole team and CI get identical environments, but you don't edit it manually — uv manages it automatically when you run `uv add`, `uv remove`, or `uv lock`.

- To create the venv using UV:

```
uv venv
```

`uv init` = to create the uv project

```
uv add api...
```

- How to delete the venv

```
rm -rf venv
```
- Clear all downloaded packages, not the venv itself

```
uv cache clean
```
- To check which packages are installed within the environment.

```
pip list
```

To delete unused packages

install pip-autoremove

Check package dependencies, version, install location..

```
pip show numpy
```

Create a project with UV

```
uv init {project_name}
```

Inside the project, there are:

- .gitignore
- .python-version
- hello.py
- pyproject.toml
- README.md

Create the environment and dependencies

```
uv add openai pydantic fastapi numpy
```

--> This will simply create the `.venv` and install the mentioned package

To remove unwanted library

```
uv remove fastapi
```

Project workflow

```
uv init my-project
cd my-project
cursor .
uv add openai fastapi
echo 'API_KEY=API_KEY' > .env

git init
git add
git commit -m 'Initial commit'

# Create new repo with Github CLI
gh repo create my-project --private --source=. --remote=origin --push
```

Run a python file (we dont have to have python in your machine)

```
uv hello.py
```

```
uv sync
```